Code Basics for Designers

HOW YOUR FIGMA DECISIONS
BECOME FRONTEND RULES

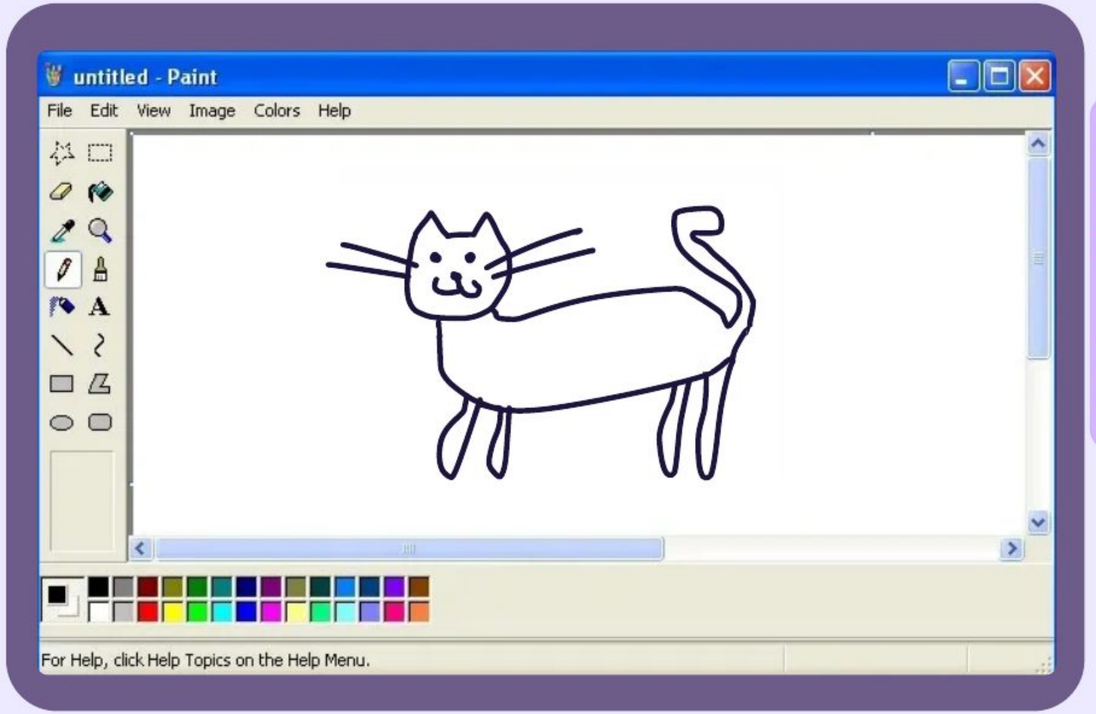
Magdalena Sapkowska
Product Designer



Nostalgia, anyone?

Before we dive into technicalities, let us start with where many interfaces began: pixels on a screen.

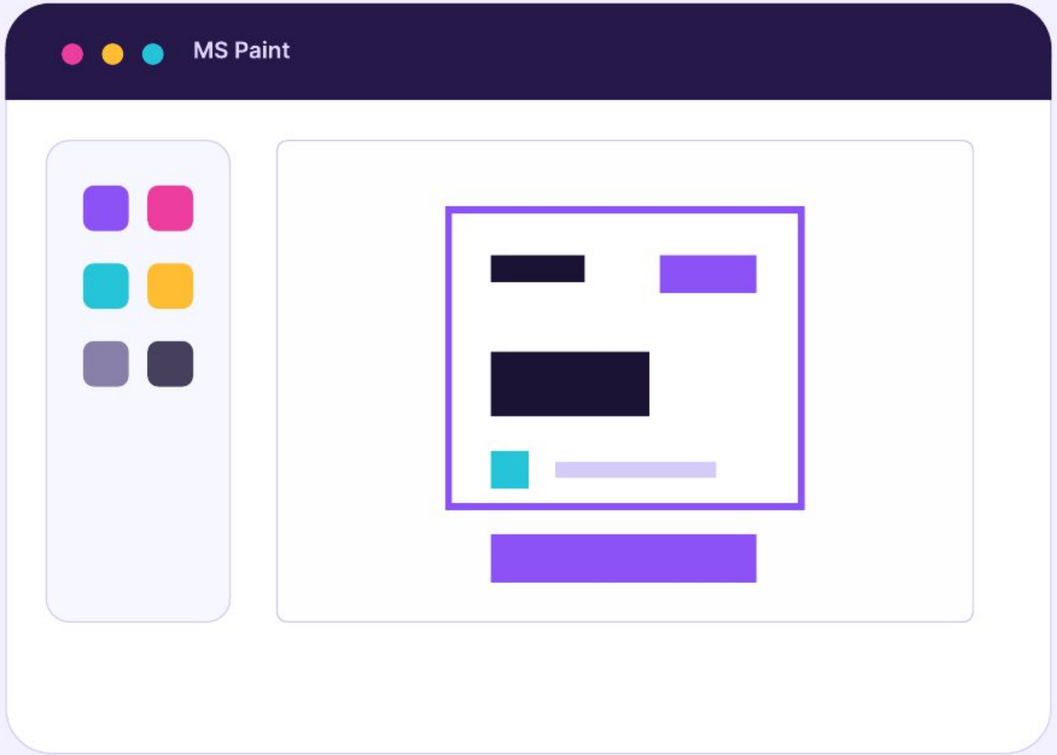
Do you remember?



We have come a long way from painting pixels.

First, interfaces were flat pictures

Paint-era design was one bitmap. If something changed, you repainted pixels.



One flat layer

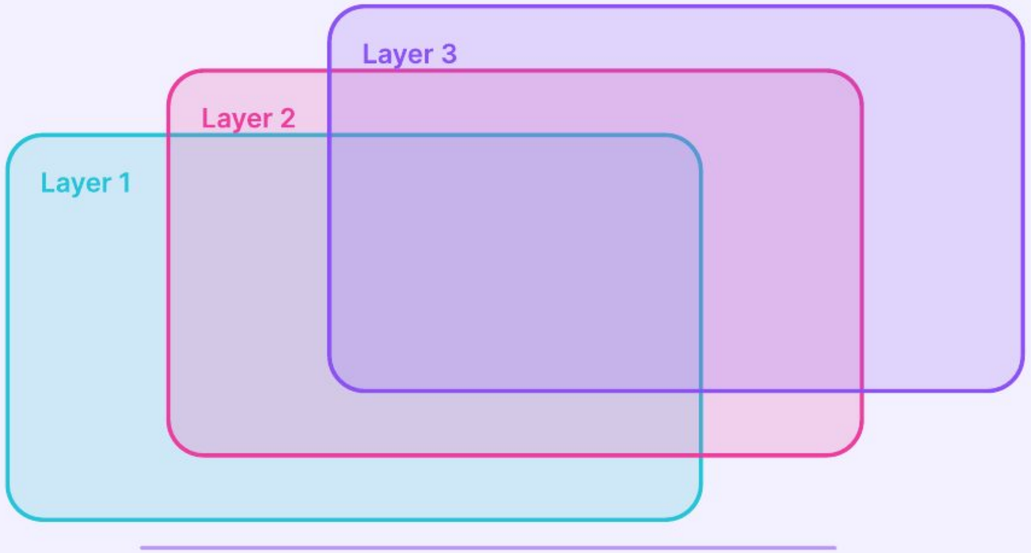
Great for drawing. Hard for explaining hierarchy, reuse, or responsive behavior.

**It captured the look,
but not the logic.**

Then design got layers

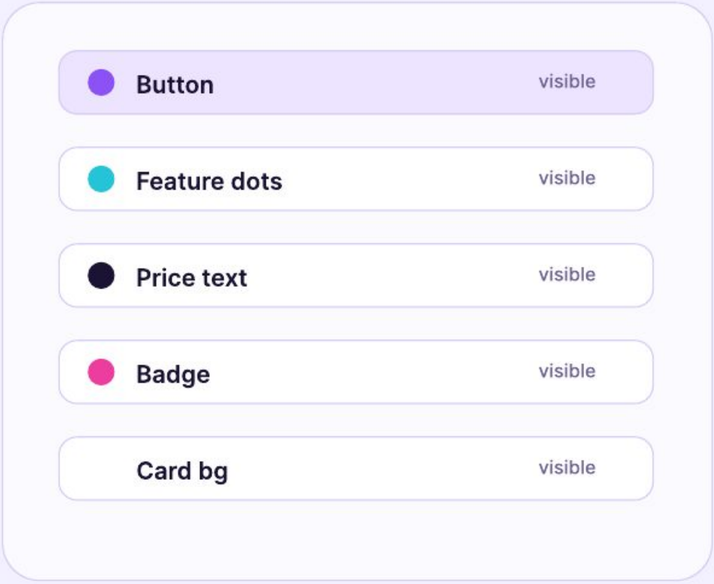
Photoshop gave us stacked objects: powerful composition, but still mostly visual ordering.

Like transparent sheets on a projector



**Visual order became editable,
but the file still did not know the product structure.**

Layers panel

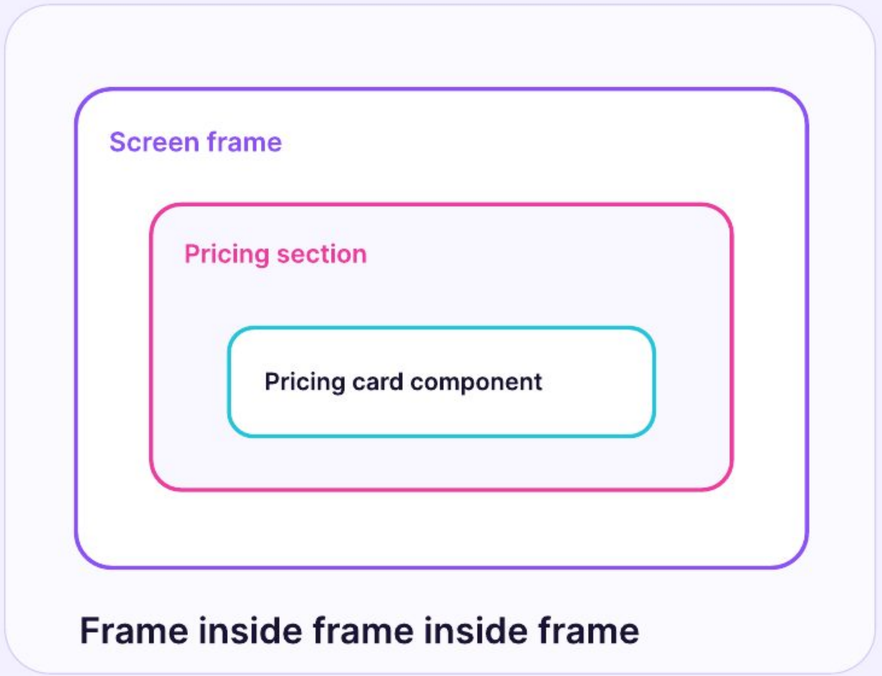


**Better: order and editability.
Still missing: containers, rules, and meaning.**

Now design has structure

Figma frames can nest like code: a screen contains sections, sections contain components.

Figma frame tree



Frame inside frame inside frame

Code hierarchy

```
<main>
  <section class="pricing">
    <PricingCard />
  </section>
</main>
```

The bridge: designers already make hierarchy. Code asks us to name it.

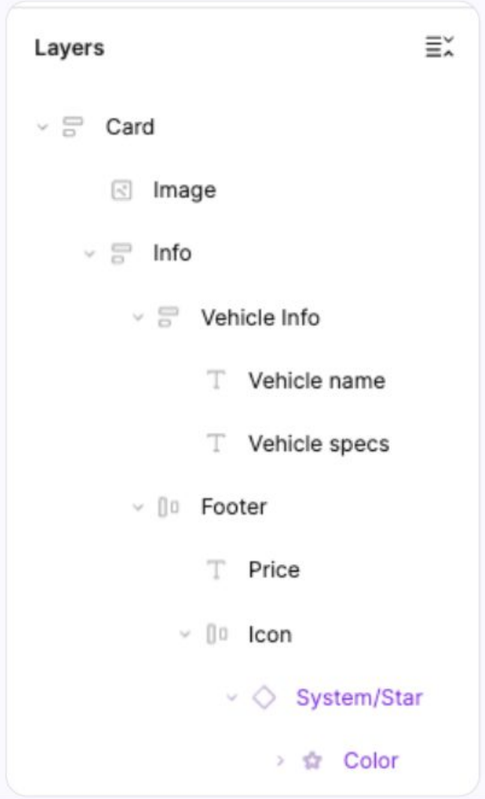
Think about Figma almost as semantic input, not merely a visual canvas.

Structure on the left, style on the right

In frontend terms: layers are HTML; properties are CSS.

Layers panel

HTML



Two panels, two jobs

The left panel tells engineers what the interface is. The right panel tells them how it should look and respond.

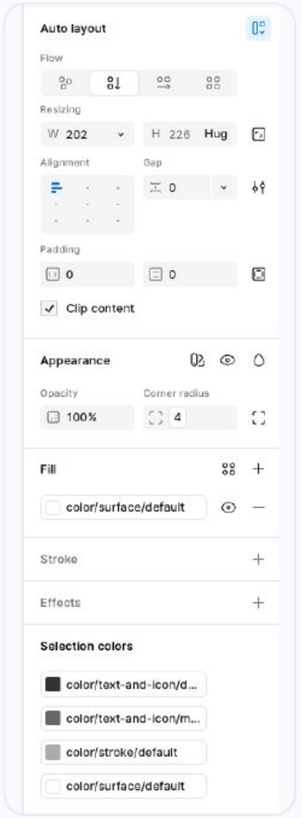
HTML Structure, hierarchy, and meaning.

CSS Layout, spacing, color, and typography.

Handoff = structure + presentation

Design properties

CSS



Layout → card → code

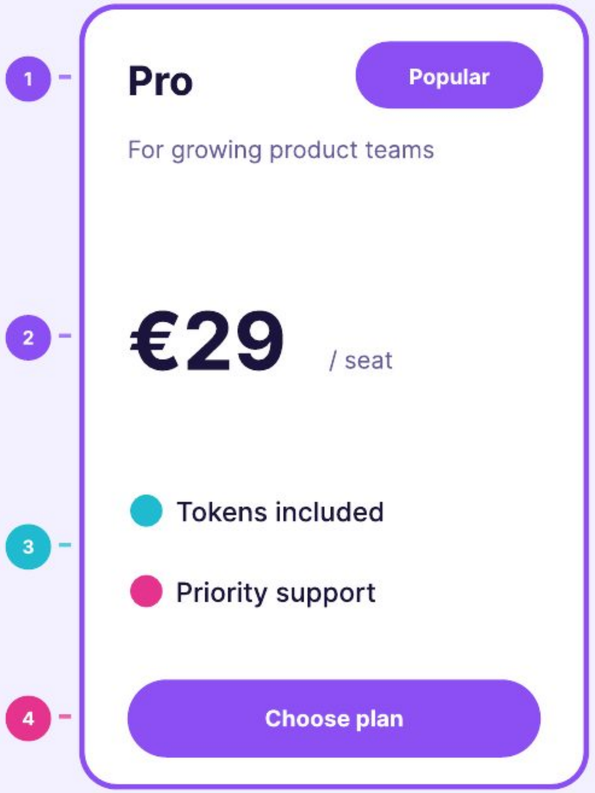
Make the structure visible once, then reuse the same pieces in design and implementation.

Figma structure

The rules should describe this exact card.

1. **Card shell**
   Column container with 32px padding
2. **Content stack**
   Header, price, features, button
3. **Repeatable row**
   Icon + label use the same component
4. **CTA area**
   Button fills the available card width

This is the reusable component model, not a one-off drawing.



CSS structure

1. **.pricing-card**
   display: flex;
   flex-direction: column;
   justify-content: space-between;
   padding: 32px;
2. **.price-row**
   display: flex;
   align-items: baseline;
   gap: 10px;
3. **.feature-row**
   display: flex;
   align-items: center;
   gap: 16px;
4. **.cta**
   width: 100%;

Each number points to one reusable piece: the design object, the card section, and the implementation rule stay aligned.

Design words have frontend twins

A tiny cheat sheet for the card we'll build.

In Figma		In code	Designer intuition
Frame	→	div / section / container	A box that holds other boxes
Auto Layout	→	Flexbox	Direction, gap, alignment, wrapping
Styles panel	→	CSS properties	Visual rules applied to elements
Variables / tokens	→	CSS variables / theme tokens	Named values reused everywhere
Component variants	→	Props / states	Reusable UI with controlled differences

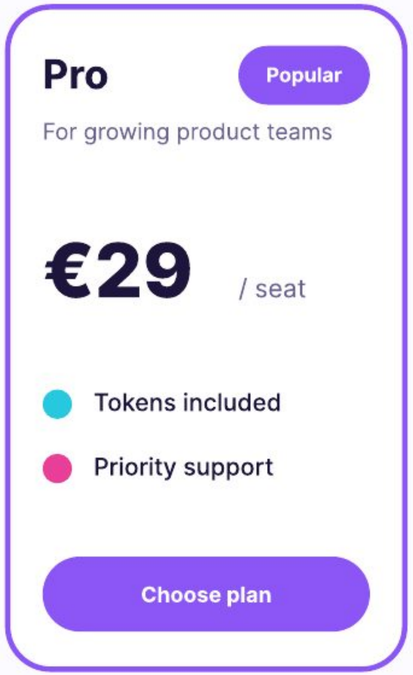
Use this as translation, not a perfect dictionary.

Step 1: Frame → container

First question: what contains what?

Figma layer tree

Frame	PricingCard
Container	Header
Text	Plan + subtitle
Group	Price
List	Features
Button	CTA



HTML structure

```
<article class="pricing-card">
  <header>
    <Badge>Popular</Badge>
    <h3>Pro</h3>
    <p>For growing teams</p>
  </header>

  <Price amount="€29" unit="seat" />
  <FeatureList />
  <Button />
</article>
```

Step 2: Auto Layout → Flexbox

Auto Layout defines how the card breathes: direction, spacing, padding, and stretch behavior.

Figma Auto Layout

Auto Layout panel

Raw layout values

Direction	column
Gap	24px
Padding	32px
Align	stretch

Variables come in the next step, after the raw spacing decision is clear.

Pro Popular

For growing product teams

€29 / seat

- Tokens included
- Priority support

Choose plan

CSS properties

```
.pricing-card {
  display: flex;
  flex-direction: column;
  gap: 24px;
  padding: 32px;
  align-items: stretch;
}

@media (max-width: 480px) {
  .pricing-card { width: 100%; }
}
```

Step 3: Styles → CSS properties

Fill, stroke, radius, effects, and type become explicit rules.

Figma styles

Style panel

Fill	#FFFFFF
Radius	34
Stroke	#8B56F5
Effect	soft
Typography	Inter

Pro Popular

For growing product teams

€29 / seat

- Tokens included
- Priority support

Choose plan

CSS properties

```
.pricing-card {
  background: #fff;
  border: 1px solid #ddd6fe;
  border-radius: 34px;
  box-shadow: 0 24px 70px rgb(91 33 182 /
.14);
}

.pricing-card h3 {
  font: var(--text-title);
}
```

Step 4: Variables → tokens

Tokens become CSS variables in code.

Figma variables

color.action	#8B5CF6
space.300	24px
radius.card	34px
text.title	34 / bold

Pro Popular

For growing product teams

€29 / seat

- Tokens included
- Priority support

Choose plan

CSS variables

```
:root {
  --color-action: #8b5cf6;
  --space-300: 24px;
  --radius-card: 34px;
  --text-title: 34px/1.1 Inter;
}

.button {
  background: var(--color-action);
}
```

Step 5: Variants → component props

Variants become the controls engineers use to reuse the component.

Figma component variants

Default

Featured

Disabled

Pro Popular

For growing product teams

€29 / seat

- Tokens included
- Priority support

Choose plan

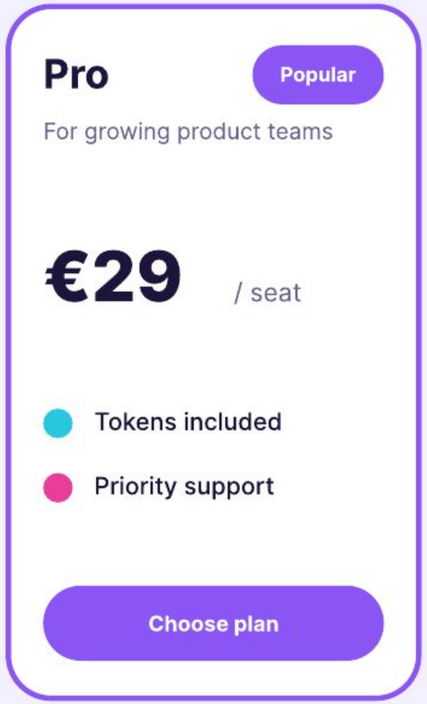
Component props

```
<PricingCard
  plan="Pro"
  price="€29"
  variant="featured"
  size="large"
  ctaLabel="Choose plan"
  features={features}
  disabled={false}
/>
```

Before handoff, define these five things

The clearer the decisions, the easier the build. A strong Figma structure gives it a much more reliable blueprint for reliable code.

- **Structure** — What are the containers?
- **Layout** — How should it respond?
- **Tokens** — Which values are reusable?
- **States** — What can change or fail?
- **Components** — Which pieces should be reused?



A well-structured Figma component with sensible Auto Layout, variants, variables, and naming gives the AI clues about what the eventual code architecture should be.

Messy Figma can contribute to messy AI-generated code.

Demo: inspect the card in Figma + code